HS PF

Programmieren in Python

Python-Grundlagen: Numpy Teil1

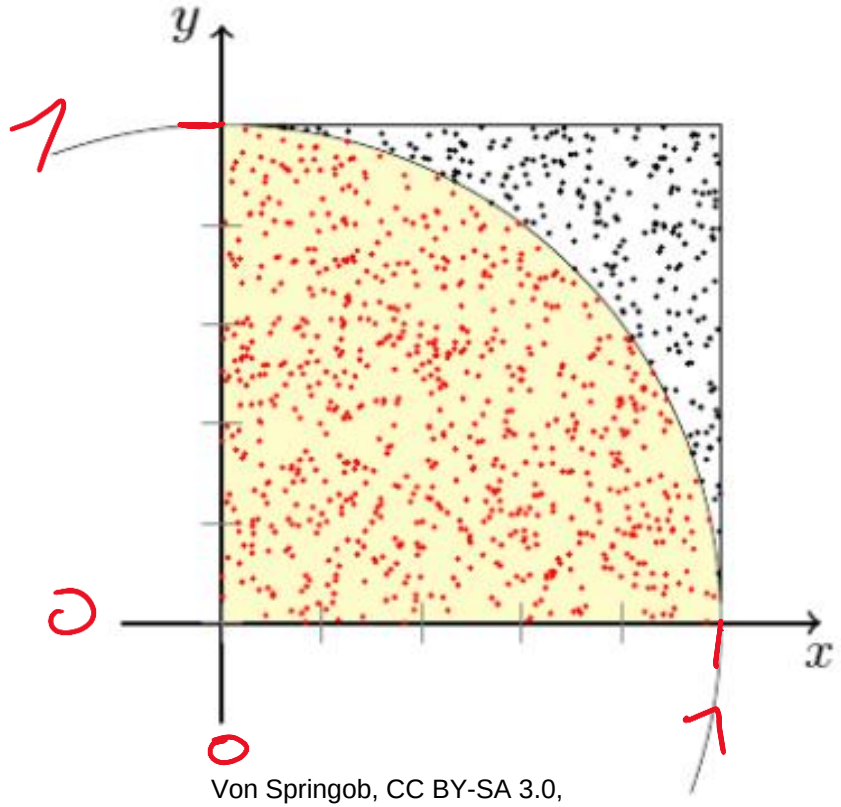
Prof. Dr.-Ing. Stefan Kray

Aufgabe montecarlo.py

- Schreiben Sie ein Programm „montecarlo.py“, dass die Zahl Pi durch eine numerische Simulation berechnet

- Vorgehen:
  - x=Zufallszahl von 0..1
  - y=Zufallszahl von 0..1
  - Die Wahrscheinlichkeit, dass x**2+y**2<1 beträgt $\frac{\pi}{4}$
  - Das Experiment muss für eine große Anzahl von Zufallszahlen gelöst werden.
  - Hinweis: Zufallszahl in Python mit:

```
import random
…
x=random.random()
```



Von Springob, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=10185824

Numpy

- Numpy ist eine numerische Bibliothek
- Sie gehört nicht zum Standardumfang von Python, ist jedoch sehr populär
- Seit 1995 in Entwicklung (noch als „Numeric“), seit 2006 als NumPy
- Datenstruktur in Numpy ist das (NumPy)-Array

Array

- NumPy-Arrays können ein- oder mehrdimensional sein
- Sie können über NumPy-Funktionen oder aus Python-Listen erstellt werden

```
import numpy as np
x=np.array([1,2,3])
```

- x ist nun ein Numpy-Array. Ein Numpy-Array ist ein Python-Objekt, dass auf die schnelle Verarbeitung von Daten ausgelegt ist
- NumPy-Arrays können in vielfältiger Weise miteinander verrechnet werden

Array-Erzeugung (1)

- NumPy-Arrays werden auf verschiedene Art erzeugt
- NumPy-Arrays können beliebige Dimensionen aufnehmen
- Aus einer **Liste**:
  - 1D: `x=np.array([1,2,3])`
  - 2D: `x=np.array([[1,2,3],[3,4,5]])`
  - …
- Menge von **Nullen oder Einsen**
  - 1D: `x=np.zeros(3)`
  - 2D: `x=np.zeros((2,3))` #zeilen,spalten
  - 3D: `x=np.ones((2,3,4))`

Array-Erzeugung (2)

- Aus Range-Objekten: `x=np.array(range(3,8))`
- Aus NumPy-**Range**-Objekten (schneller!): `x=np.arange(3,8)`

- Linear verteilte Zahlen mit „**linspace**“ (wie in Matlab):
- Parameter: Start-Zahl, End-Zahl, Anzahl der Elemente

```
np.linspace(1,4,num=10,endpoint=True)
```

- Ergebnis: array([1. , 1.33333333, 1.66666667, 2. , 2.33333333, 2.66666667, 3. , 3.33333333, 3.66666667, 4.])
- Der Parameter “Endpoint” gibt an ob der Endwert – hier die 4 – noch dazugehören soll
- Mit endpoint**=False**: array([1. , 1.3, 1.6, 1.9, 2.2, 2.5, 2.8, 3.1, 3.4, 3.7])

- weitere: np.indices

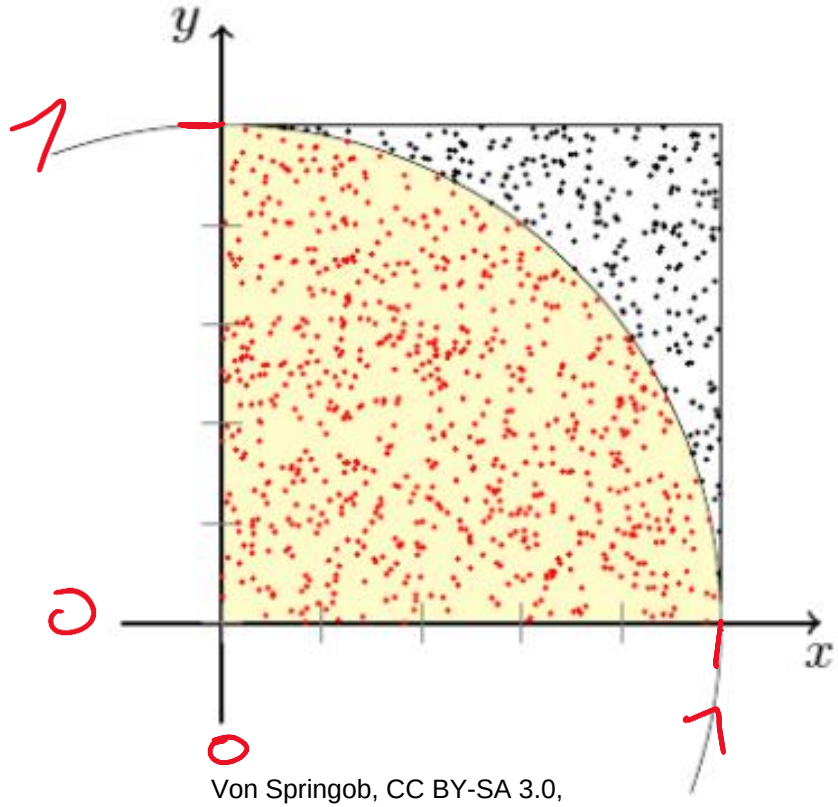
Array-Erzeugung (3)

- Aus Zufallszahlen (**np.random**)
- `np.random.rand(10)` → erzeugt 10 Zufallszahlen von 0..1
- `np.random.rand(10,20)` → erzeugt eine Matrix mit 10x20 (Zeilen x Spalten) Zufallszahlen von 0..1
- weitere Zufallszahlen:
  - `np.random.randint` → integer Zufallszahlen
  - `np.random.choice` → Zufällige Elemente aus Array auswählen
  - siehe https://numpy.org/doc/stable/reference/random/index.html

- Tipp: Abfrage der Array-Größe mit .shape (ohne Klammer!)

Aufgabe montecarlo_np.py

- Schreiben Sie ein Programm „montecarlo_np.py“, dass die Zahl Pi durch eine numerische Simulation berechnet
- Verwenden Sie jetzt **NumPy**
- Starten sie mit: `import numpy as np`
- Hinweis:
  - die Summe eines NumPy-Array kann mit `np`.sum(…) berechnet werden. Das funktioniert auf für Boolean-Arrays.



Von Springob, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=10185824

Array-Erzeugung (4)

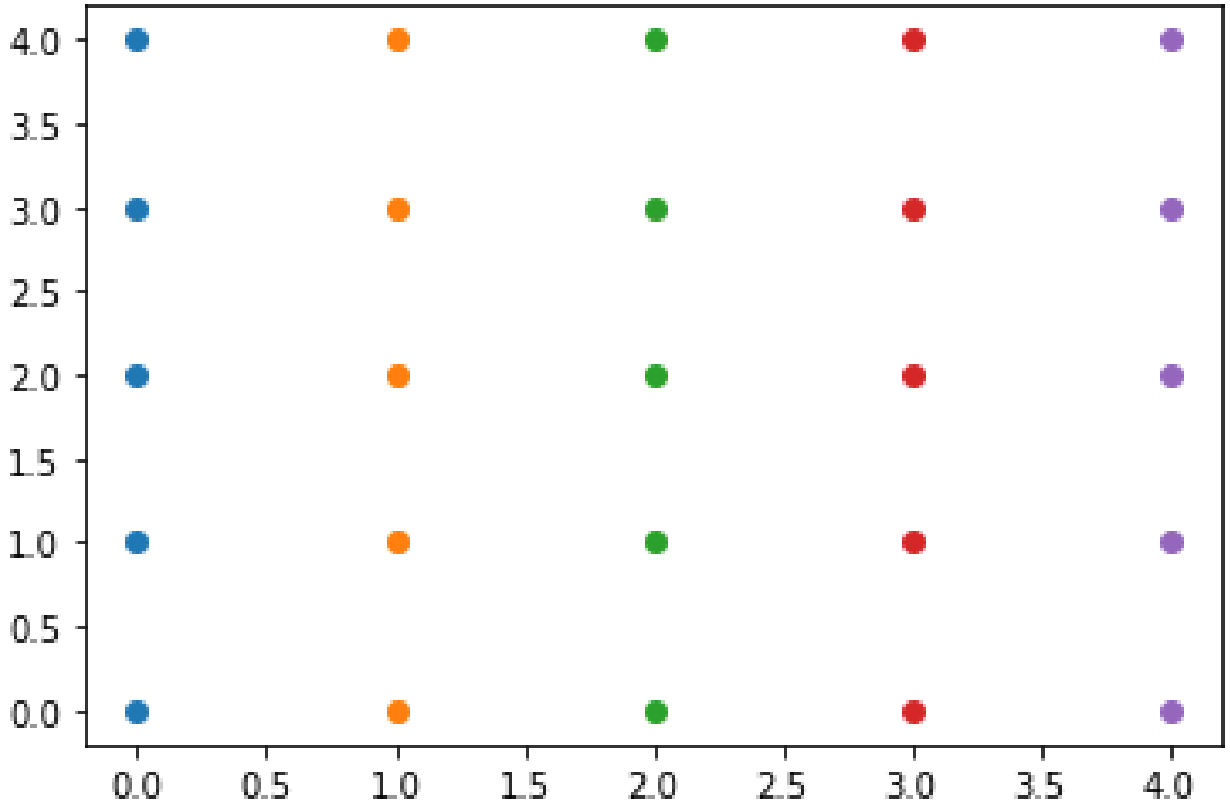
- np.meshgrid: 2D-Array aus zwei 1D-Arrays erzeugen
- Beispiel:

```
xvalues = np.array([0, 1, 2, 3, 4])
yvalues = np.array([0, 1, 2, 3, 4])
xx, yy = np.meshgrid(xvalues, yvalues)
```

```
xx
Out[107]:
array([[0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4]])

yy
Out[108]:
array([[0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [2, 2, 2, 2, 2],
       [3, 3, 3, 3, 3],
       [4, 4, 4, 4, 4]])
```

xx,yy sind 2D-Arrays
plot(xx, yy, ‚o‘)



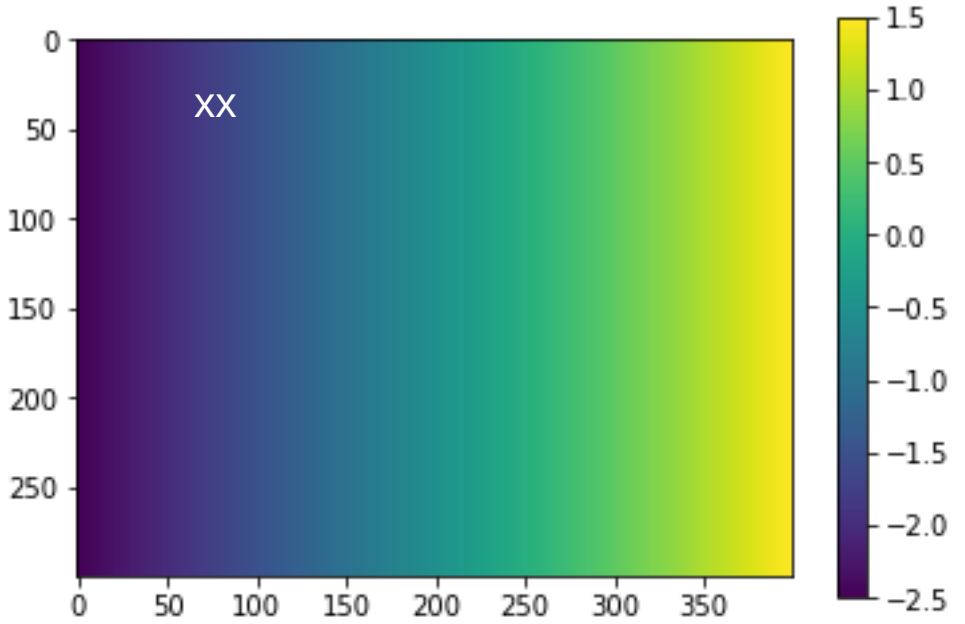
meshgrid vs. mgrid

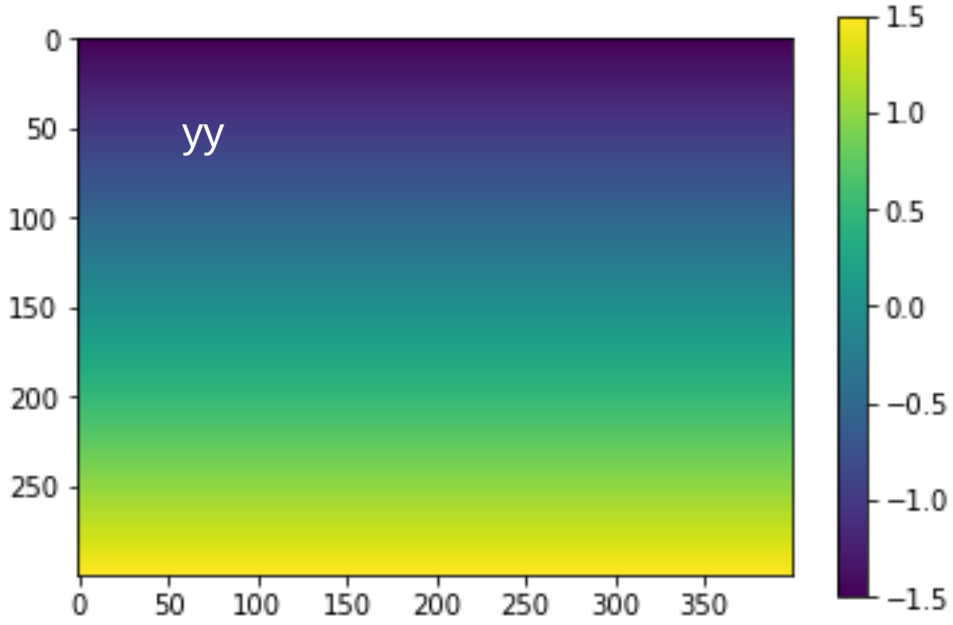
```
x = np.linspace(-2.5, 1.5, 400)
y = np.linspace(-1.5, 1.5, 300)
xx, yy = np.meshgrid(x, y)
```

- **Kurzform mit mgrid**

```
yy, xx = np.mgrid[-1.5:1.5:300j,-2.5:1.5:400j]
```

- y,x – Reihenfolge gegenüber meshgrid vertauscht
- Keine Funktion! Eckige Klammer!
- Schreibweise ...:300j bzw ...:400j = Anzahl der Elemente. Wird hier durch das kleine “j” markiert (komplexe Zahl als Marker).





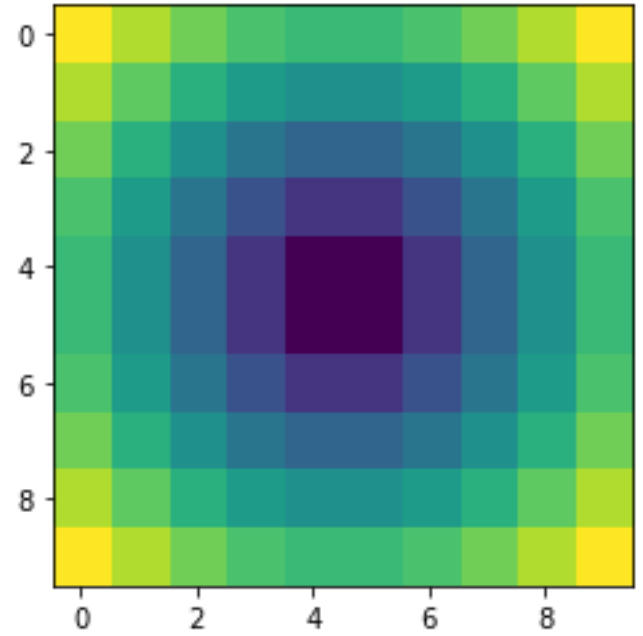
Beispiel Meshgrid:

```
import numpy as np
import matplotlib.pyplot as plt

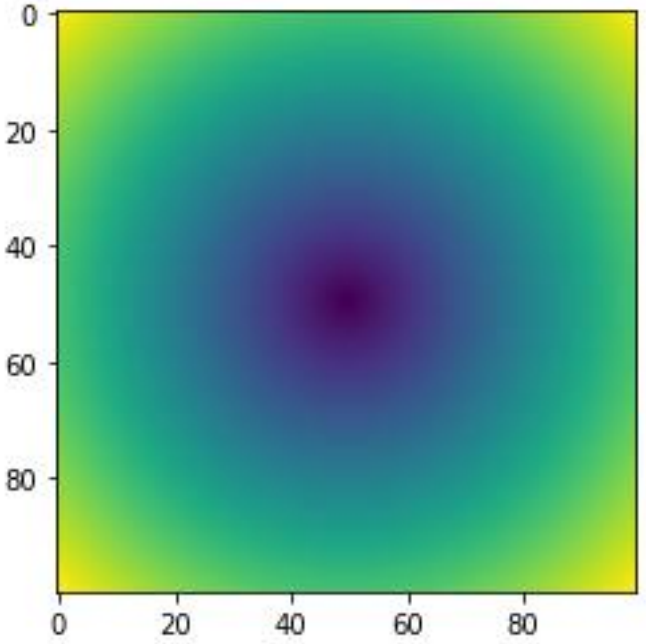
N=10
x = np.linspace(-5, 5, N)
y = np.linspace(-5, 5, N)
xx, yy = np.meshgrid(x, y)
zz = np.sqrt(xx**2 + yy**2)

plt.imshow(zz)
```

N=10



N=100



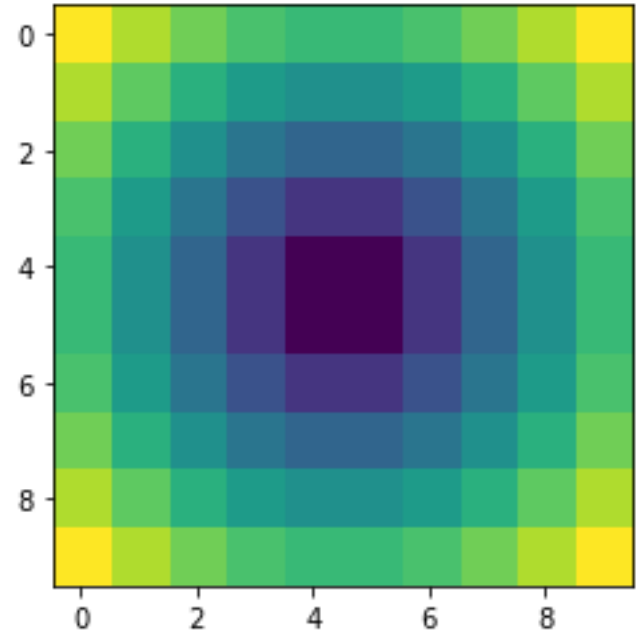
Kurzform mit mgrid:

```
import numpy as np
import matplotlib.pyplot as plt

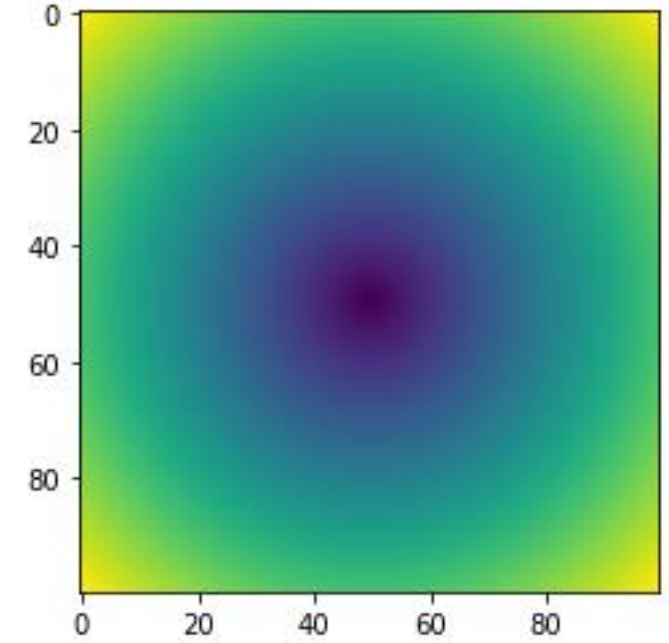
xx, yy = np.mgrid[-5:5:100j,-5:5:100j]
zz = np.sqrt(xx**2 + yy**2)

plt.imshow(zz)
```

N=10



N=100



Aufgabe: mgrid.py

- Importieren Sie numpy und matplotlib:

```
import numpy as np
import matplotlib.pyplot as plt
```

- Teil1:
  - Erstellen Sie ein mgrid von y=-1,5..1,5 (300 Werte) und x=-2,5..1,5 (400 Werte)
  - Erstellen Sie ein komplexes 2D-Array C, dass den Realteil xx und den Imaginärteil yy hat.
  - Geben Sie |C| mit plt.imshow() aus mit `plt.imshow(np.abs(C))`

Aufgabe: mgrid.py

- Importieren Sie numpy und matplotlib:

```
import numpy as np
import matplotlib.pyplot as plt
```

- Teil1:
  - Erstellen Sie ein mgrid von y=-1,5..1,5 (300 Werte) und x=-2,5..1,5 (400 Werte)
  - Erstellen Sie ein komplexes 2D-Array C, dass den Realteil xx und den Imaginärteil yy hat.
  - Geben Sie |C| mit plt.imshow() aus mit `plt.imshow(np.abs(C))`

- Teil2:
  - Erstellen Sie ein 2D-Array Z mit der Größe von C (300x400). Initialisieren Sie es zu Null.
  - Berechnen Sie 20 Mal: Z=Z**2+C
  - Geben Sie mit plt.imshow() alle Punkte von Z aus, für die gilt: |Z|<2.5

Array-Operationen

- NumPy erlaubt das Anwenden von Operationen auf ganze Arrays
- Beispiel:
  - Addition, Subtraktion, Multiplikation, Division von Konstanten
  - Vergleiche mit Zahl (>,>=,<,<=)

  - `np.arange(3)` → array([0,1,2])
  - `np.arange(3)+3` → array([3,4,5])
  - `np.arange(3)*2` → array([0,2,4])
  - `np.arange(3)<=1` → array([True, True, False])

  - Auch zwei Array gleicher Länge können elementweise addiert / subtrahiert / multipliziert / dividiert werden

- `np.arange(3)+np.arange(3)*2` → `np.array([0,3,6])`

NumPy-Datentypen (1)

- Numpy hat eigene Datentypen
- Es gibt plattformabhängige und plattformunabhöngige Datentypen
- Wichtige plattformabhängige Datentypen, analog zu C/C++:
  - byte
  - short
  - int
  - single
  - double
  - longlong

NumPy-Datentypen (2)

- Plattformunabhängige Datentypen geben ihr Format und Größe an
- Größe steht mit Anzahl Bits im Namen
- Wichtige plattformunabhängige Datentypen:
  - ganzzahliger Integer-Datentypen: int8, int16, int32, int64
  - nicht negative Integer-Datentypen: uint8, uint16, uint32, uint64
  - Float-Datentypen: float32, float64
  - Werden häufig zum Laden und Speichern von Daten verwendet

- Beispiel:

```
a=np.float32(3.0)
print(type(a))
```

  - Ergebnis: <class 'numpy.float32'>

NumPy-Datentypen (3)

- Beispiel: Array vom Typ Int32 erzeugen
- `x=np.array([1.0,2.0,3.0,4.3],dtype=np.int32)`
  - **→ array([1, 2, 3, 4])**

- Umwandeln in anderen Datentyp: .astype()
- `y=x.astype(np.float32)`
- **→ array([1., 2., 3., 4.], dtype=float32)**

Matplotlib

- Matplotlib dient zum graphischen Darstellen von Daten
- Python-Schnittstelle nennt sich „pyplot“:
- Häufig genutzter Import:
- `import matplotlib.pyplot as plt`

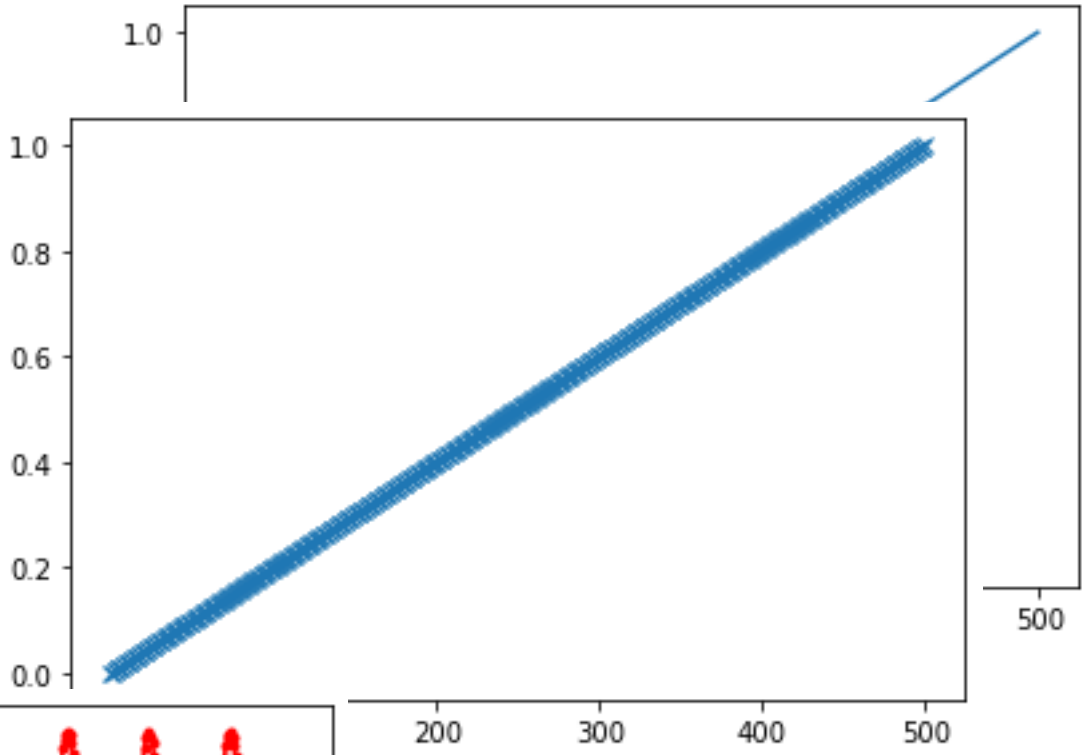
Matplotlib: plot

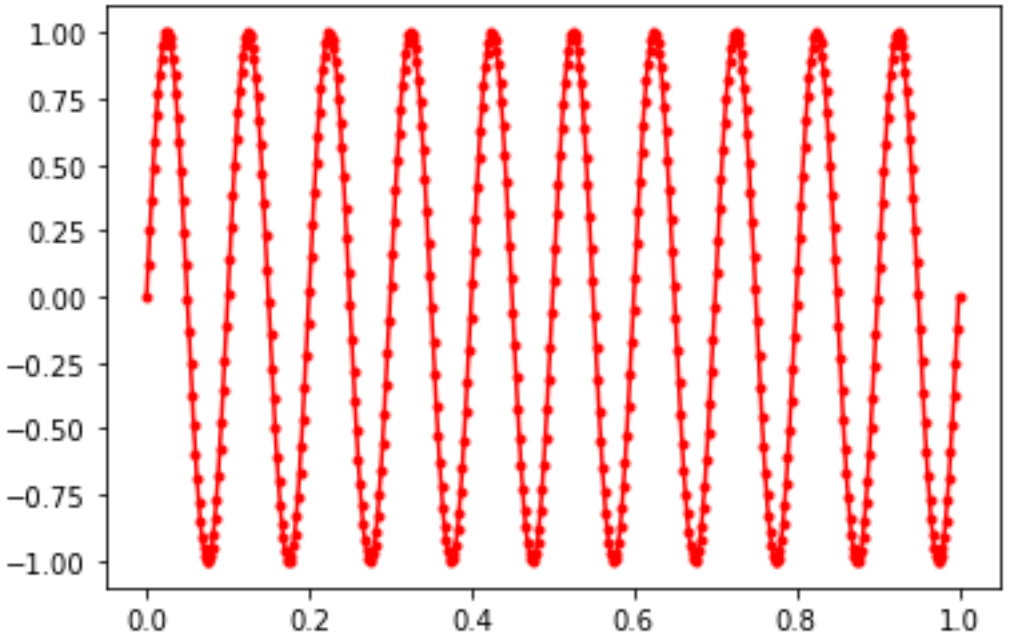
- Plot-Befehl plottet x oder x&y-Diagramme
- Beispiel 1:

```
t=np.linspace(0,1,500)
y=np.sin(2*np.pi*10*t)
plt.plot(t)
plt.plot(t,y)
```

- Beispiel 2:

```
plt.plot(t,y,'.r-')
```





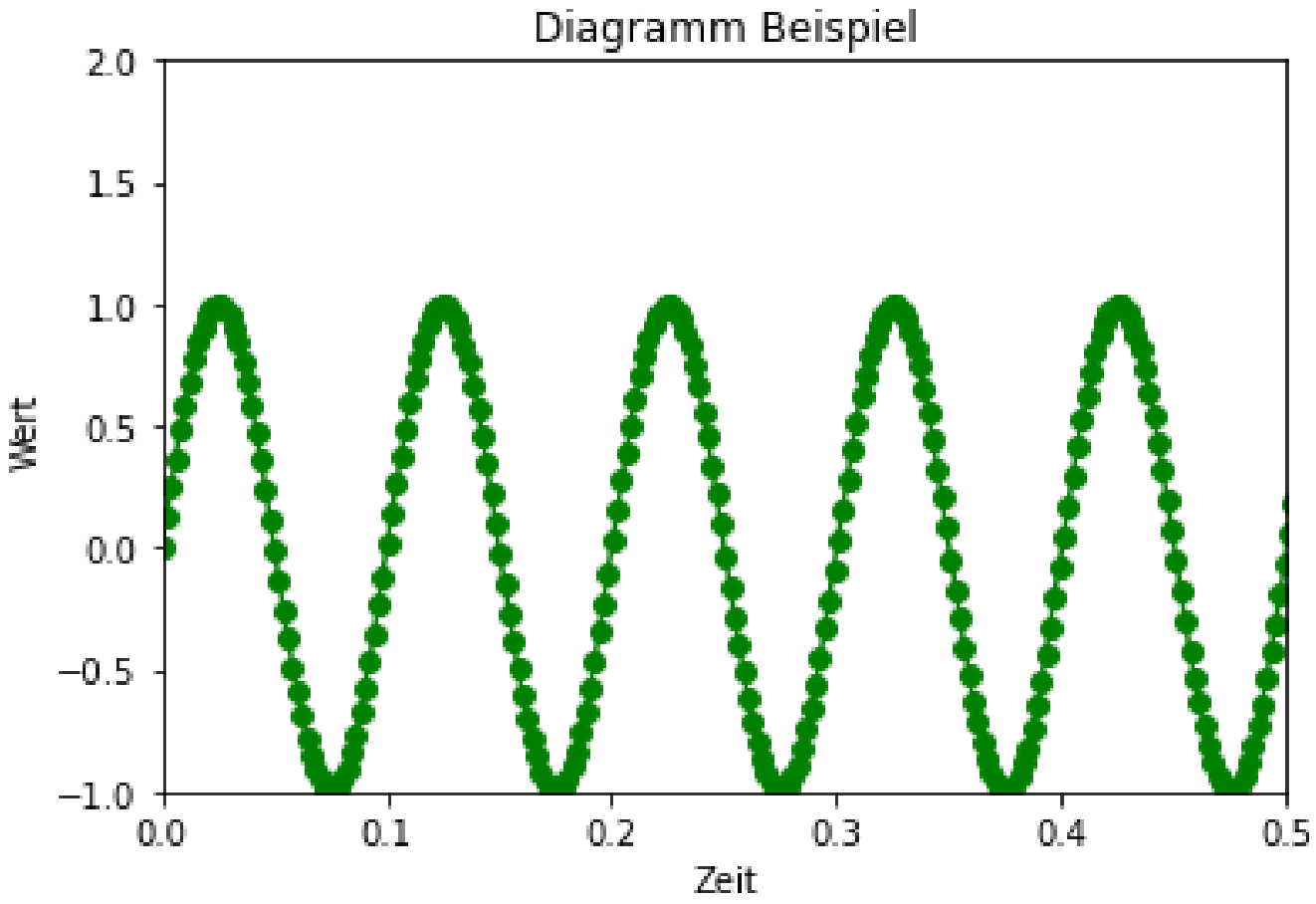
Matplotlib: plot (2)

- Beispiel: Achsenbeschriftung und Formatierung

```
import numpy as np
import matplotlib.pyplot as plt

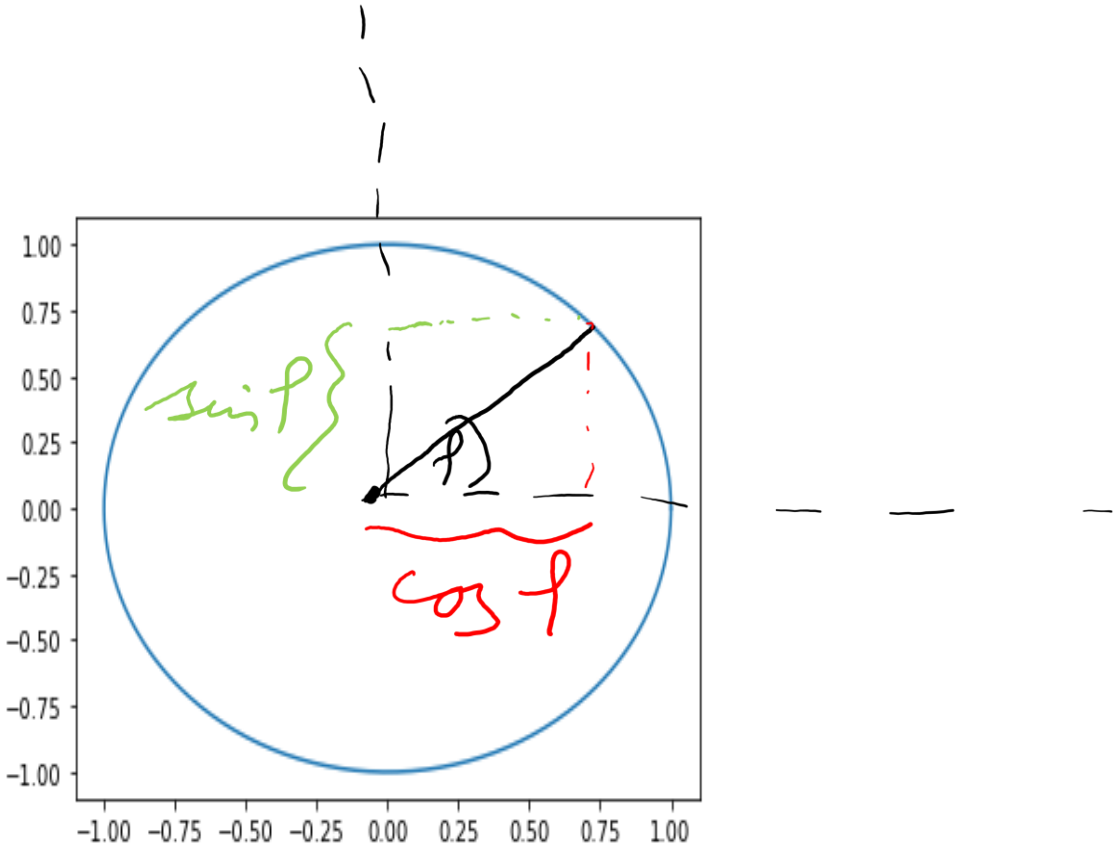
t=np.linspace(0,1,500)
y=np.sin(2*np.pi*10*t)

plt.plot(t,y,'og-')
plt.xlabel("Zeit")
plt.ylabel("Wert")
plt.title("Diagramm Beispiel")
plt.axis([0, 0.5, -1, 2])
```



Aufgabe: plot_kreis.py

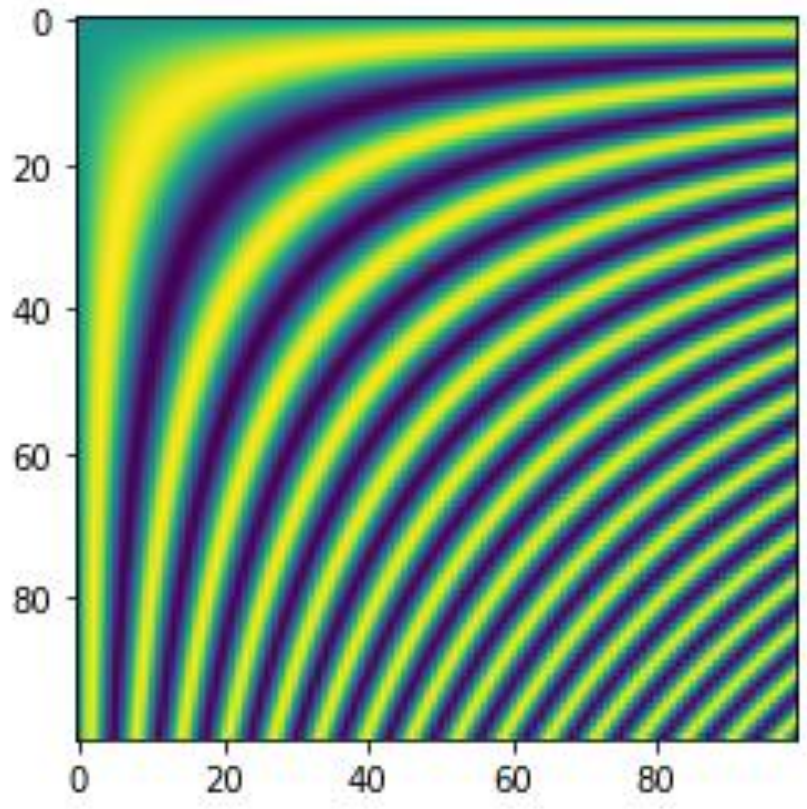
- Schreiben Sie ein Programm plot_kreis.py
- Das Programm soll mit Matplotlib und Numpy einen Kreis zeichnen
- Mittelpunkt =(0,0), Radius=1



Matplotlib: imshow

- 2D-Arrays / Bilder können mit plt.imshow() ausgegeben werden
- Beispiel:

```
img=np.zeros((100,100))
for y in range(0,100):
    for x in range(0,100):
        img[y,x]=np.sin(x*y*0.01)
plt.imshow(img)
```



Aufgabe: imshow_np.py

- Dieses Bild ohne zwei FOR-Schleifen erzeugen
- Hinweis:
  - Zwei Arrays mit numpy erstellen
  - np.meshgrid benutzen

